Nested and Wrapper Class Assignment with Polymorphism

Automobile.java:

```
public class Automobile {

  // Wrapper classes
  private Integer establishedYear;
  private Double price;
  private String slogan;
  private Character type;

  public Automobile(Integer establishedYear, Double price, String slogan, Character type) {
    this.establishedYear = establishedYear;
    this.price = price;
    this.slogan = slogan;
    this.type = type;
  }

  // Getters and Setters
  public Integer getEstablishedYear() {
    return establishedYear;
  }

  public void setEstablishedYear(Integer establishedYear) {
    this.establishedYear = establishedYear;
  }

  public Double getPrice() {
    return price;
  }

  public void setPrice(Double price) {
    this.price = price;
  }

  public String getSlogan() {
    return slogan;
  }
```

```
    public void setSlogan(String slogan) {
        this.slogan = slogan;
    }

    public Character getType() {
        return type;
    }

    public void setType(Character type) {
        this.type = type;
    }

    // sound method
    public void sound() {
        System.out.println("Automobile sound: vrooom");
    }

    // Method to print all information
    public void printInfo() {
        String name = this.getClass().getSimpleName();
        String typeText = (type == null) ? "na" : type.toString();

        System.out.println(
            name +
            " -> Year: " + establishedYear +
            ", Price: $" + price +
            ", Slogan: " + slogan +
            ", Type: " + typeText
            );
        }

        // Overloaded method
        public void printInfo(String label) {
            System.out.println(label);
            printInfo();
    }
}
```

Toyota.java:

```
public class Toyota extends Automobile {

  public Toyota() {
    // type = na → represented as null
    super(1937, 0.0, "Let's Go Places", null);
  }

  @Override
  public void sound() {
    System.out.println("Toyota sound: hummm");
  }
}
```

Corolla.java:

```
public class Corolla extends Toyota {

  public Corolla() {
    setEstablishedYear(2023);
    setPrice(21700.0);
    setType('c');
  }

  @Override
  public void sound() {
    System.out.println("Corolla sound: brrrrrr");
  }

  // Extra method
  public Integer fuelEfficiency() {
    return 32; // miles per gallon
  }
}
```

Tundra.java:

```
public class Tundra extends Toyota {
```

```
  public Tundra() {
    setEstablishedYear(2023);
    setPrice(38965.0);
    setType('t');
  }

  @Override
  public void sound() {
    System.out.println("Tundra sound: grrrrr");
  }
}
```

Highlander.java:

```
public class Highlander extends Toyota {

  public Highlander() {
    setEstablishedYear(2023);
    setPrice(36620.0);
    setType('s');
  }

  @Override
  public void sound() {
    System.out.println("Highlander sound: shhhh");
  }

  // Inner / Nested class
  class ThirdRow {
    private Integer seats;

    public ThirdRow(Integer seats) {
      this.seats = (seats == 7 || seats == 8) ? seats : 8;
    }

    public Integer getSeats() {
      return seats;
    }
```

```
  }
}
```

Main.java:

```
public class Main {

  public static void main(String[] args) {
    Automobile auto = new Automobile(1886, 9999.0, "Original car", 'a');
    Corolla corolla = new Corolla();
    Toyota toyota = new Toyota();
    Tundra tundra = new Tundra();
    Highlander highlander = new Highlander();

    // Polymorphism (dynamic binding)
    System.out.println("--- Sounds (Dynamic Binding / Polymorphism) ---");

    Automobile[] cars = {
        auto, toyota, corolla, tundra, highlander
    };

    for (Automobile car : cars) {
      car.sound();
    }

    System.out.println();
    System.out.println("--- Car Information ---");

    for (Automobile car : cars) {
      car.printInfo();
    }

    System.out.println();
    System.out.println("--- Static Binding demonstration (Overloading) ---");

    // Static binding: overloaded method selection
    auto.printInfo("This is the overloaded printInfo(String) method:");

    // Inner class demonstration
```

```
        System.out.println();
        System.out.println("--- Inner Class Demo ---");
        Highlander.ThirdRow seats7 = highlander.new ThirdRow(7);
        Highlander.ThirdRow seats8 = highlander.new ThirdRow(8);

        System.out.println("Highlander third row option: " + seats7.getSeats() + " seats");
        System.out.println("Highlander third row option: " + seats8.getSeats() + " seats");

        System.out.println();

        // Wrapper class + Corolla extra method
        System.out.println("--- Corolla extra method ---");
        Integer mpg = corolla.fuelEfficiency();
        System.out.println("Corolla fuel efficiency: " + mpg + " MPG");
        System.out.println("Estimated range for 10 gallons: " + (mpg * 10) + " miles");

        System.out.println();
    }
}
```

Output:

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBra
--- Sounds (Dynamic Binding / Polymorphism) ---
Automobile sound: vrooom
Toyota sound: hummm
Corolla sound: brrrrrr
Tundra sound: grrrrr
Highlander sound: shhhh

--- Car Information ---
Automobile -> Year: 1886, Price: $9999.0, Slogan: Original car, Type: a
Toyota -> Year: 1937, Price: $0.0, Slogan: Let's Go Places, Type: na
Corolla -> Year: 2023, Price: $21700.0, Slogan: Let's Go Places, Type: c
Tundra -> Year: 2023, Price: $38965.0, Slogan: Let's Go Places, Type: t
Highlander -> Year: 2023, Price: $36620.0, Slogan: Let's Go Places, Type: s

--- Static Binding demonstration (Overloading) ---
This is the overloaded printInfo(String) method:
Automobile -> Year: 1886, Price: $9999.0, Slogan: Original car, Type: a

--- Inner Class Demo ---
Highlander third row option: 7 seats
Highlander third row option: 8 seats

--- Corolla extra method ---
Corolla fuel efficiency: 32 MPG
Estimated range for 10 gallons: 320 miles


Process finished with exit code 0
```